AI INFRA RESEARCH SERIES

# KDA / Kimi Linear

从 Delta Rule 到可并行的混合线性注意力

**分析对象**

Kimi Linear: An Expressive, Efficient Attention Architecture

**作者与机构**

Kimi Team；Moonshot AI（月之暗面）。

**版本范围**

arXiv:2510.26692v2。首版 2025-10-30、v2 2025-11-01；KDA 是这篇报告的核心机制，不是另造的 2026 年论文。另以 Kimi K3 v2 说明后续改动。

**核心判断**

KDA 用通道级遗忘和 Delta Rule 管理矩阵状态，再把逐 token 递归改写成 chunk 内矩阵运算。Kimi Linear 通过 3:1 的 KDA/MLA 混合兼顾速度与全局读取；它没有消除有限状态的信息容量限制，也不等于把任意旧模型无损替换成线性注意力。

阅读定位：机制与公式、系统实现、实验口径、局限与复现。

说明：性能数据均标明来源；本报告没有独立训练模型或复跑 GPU 性能。推导与教学算例不作为作者实测结果。

整理日期：2026 年 10 月 31 日

# 阅读导航

1 KDA、Kimi Linear、Kimi K3 是什么关系 2

2 从普通线性注意力推到 Delta Rule 2

3 KDA 的核心：不同通道独立遗忘 3

4 它与一般 DPLR 有何区别 4

5 最重要的系统问题：递归怎么在 GPU 上并行 4

6 数值难点与 K3 的后续修改 5

7 混合架构为何仍保留 MLA 6

8 显存与速度：75% 和 6× 该怎么理解 6

9 质量实验：并非每一列都领先 6

10 从状态缓存到推测解码：新的运行时约束 7

11 面向算子开发的复现与研究计划 7

12 最后总结与原始资料 8

文中的 [S1]、[S2] 等编号指向文末原始资料。引用附带章节、表号或源码位置，便于核对；不同报告的编号独立。

# 1 KDA、Kimi Linear、Kimi K3 是什么关系

**KDA** 是 Kimi Delta Attention，一种带通道级遗忘的 delta-rule 线性注意力模块。**Kimi Linear** 是以 KDA 与 MLA 混合构成的架构及模型报告。**Kimi K3** 是后续更大模型，复用这条设计路线，并修改了 KDA 的数值参数化和输出 gate。[S1；S2]

原 Kimi Linear 报告的模型规模为 48B 总参数、3B 激活参数，采用三层 KDA 加一层 MLA 的 layerwise hybrid。其公平对比主要使用相同训练 recipe 和 1.4T tokens；发布 checkpoint 的训练与后训练设置需要另行区分，不能把不同训练预算的模型拿来证明纯架构收益。[S1，§4–5]

最直观的理解是：多数层用一个固定大小的 “可更新记忆矩阵” 处理历史，少数 MLA 层继续显式访问全部历史内容。它没有把整个模型变成零 KV Cache 系统。

# 2 从普通线性注意力推到 Delta Rule

## 2.1 加法记忆会把关联不断叠上去

设 $k_t, q_t \in \mathbb{R}^{d_k}$，$v_t \in \mathbb{R}^{d_v}$，状态 $S_t \in \mathbb{R}^{d_k \times d_v}$。最简单的关联记忆是

$$S_t = S_{t-1} + k_t v_t^T, \qquad o_t = S_t^T q_t.$$

它把历史 key-value 外积加进一个矩阵。读取时用 q 寻址。优点是 decode 的状态大小不随历史长度增加；缺点是多个相似 key 会干扰，同一关联更新时也没有显式纠正旧值。[S1，§2.2]

这里的 “linear” 主要指在固定特征维度下, 序列长度上的计算可线性增长, 不表示它一定与 softmax attention 完全等价。

## 2.2 Delta Rule 把 “追加” 变成 “纠错”

考虑当前关联的重构误差

$$\mathcal{L}_t(S) = \frac{1}{2}\|S^T k_t - v_t\|_2^2.$$

对 S 做一步梯度下降:

$$S_t = S_{t-1} - \beta_t k_t (k_t^T S_{t-1} - v_t^T),$$

即

$$S_t = (I - \beta_t k_t k_t^T) S_{t-1} + \beta_t k_t v_t^T.$$

它先看当前 state 对 key 的预测，再写入预测误差。相似 key 对应的旧关联可以被纠正，而不是无限增加一份重复值。[S1，§2.2]

$\beta_t$ 类似学习率，但这是前向模型内部的 fast-weight 更新，不是优化器对模型参数的 SGD。两个 “学习过程” 处在不同层次。

### 2.3 Gated DeltaNet 增加统一遗忘

GDN 在一个 head 内使用标量 $\alpha_t$：

$$S_t = \alpha_t(I - \beta_t k_t k_t^T)S_{t-1} + \beta_t k_t v_t^T.$$

这能衰减过时信息，却让这个 head 的全部通道采用相同保留比例。若一部分通道承载长期事实、另一部分承载短期状态，统一遗忘会限制表达灵活性。[S1]

## 3 KDA 的核心：不同通道独立遗忘

KDA 将标量换为对角矩阵 $D_t = \mathrm{Diag}(\alpha_t)$：

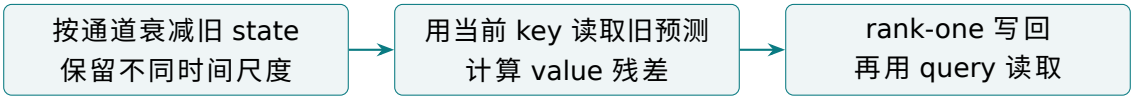
$$\boxed{S_t = (I - \beta_t k_t k_t^T)D_t S_{t-1} + \beta_t k_t v_t^T}$$

$$o_t = S_t^T q_t.$$

操作顺序为：

$$\bar{S}_t = D_t S_{t-1}, \quad \hat{v}_t = \bar{S}_t^T k_t, \quad S_t = \bar{S}_t + \beta_t k_t (v_t - \hat{v}_t)^T.$$

这是论文式 1 的等价拆解。[S1，§3] $\alpha_t$ 控制 key 通道方向上的状态保留，$\beta_t$ 控制当前关联更新强度。value 通道并没有凭这个对角门变成完全独立的另一套衰减。




### 3.1 一个二维数值例子

设旧状态为 $\mathrm{Diag}(10, 1)$，$\alpha = (0.99, 0.1)$，$k = (0, 1)^T$，$v = (0, 3)^T$，$\beta = 0.5$。衰减后是 $\mathrm{Diag}(9.9, 0.1)$，当前 key 读出 $(0, 0.1)$，写回残差的一半后得到 $\mathrm{Diag}(9.9, 1.55)$。

第一通道长期信息几乎保留，第二通道旧值先忘记，再向新关联 3 靠近。这个例子只是解释，不是训练结果。

### 3.2 顺序不可交换

标量 $\alpha I$ 与其他矩阵可交换，但一般对角矩阵不能：

$$(I - \beta k k^T)D \neq D(I - \beta k k^T).$$

把 gate 位置移错，会改变递归而不只是改变浮点舍入。这一点对自己写 CUDA/Triton/TileLang kernel 尤其重要。

# 4 它与一般 DPLR 有何区别

展开转移矩阵：

$$(I - \beta kk^T)D = D - \beta k(k^T D).$$

因此它属于 diagonal-plus-low-rank 类结构，但低秩两侧与 k、D 绑定，不是任意独立向量的通用 DPLR。限制结构使数学更贴近 delta rule，也减少了通用实现中的若干矩阵计算与数值处理。[S1，§3.2、§6.2]

“表达更灵活” 与 “执行更规则” 不是同一方向：把所有因子独立参数化，可能增加表达空间，也可能使硬件成本明显上升。论文的设计选择是在已有很强的结构里增加通道级遗忘，而不追求完全一般的状态转移。

q/k 的 L2 归一化也有稳定性意义。若 $\|k\|_2 = 1$、$0 \le \beta \le 1$，$I - \beta kk^T$ 在 k 方向的特征值为 $1 - \beta$，正交方向为 1。再乘 $D$ 后，旧状态差异满足

$$\|\Delta S_t\|_F \le \max_j \alpha_{t,j} \, \|\Delta S_{t-1}\|_F.$$

这是本报告的稳定性解释。它说明旧状态传播不会因该线性部分任意放大；却不保证整个神经网络永不爆炸，因为新的写入、投影、归一化与长链反向仍需要分析。

# 5 最重要的系统问题：递归怎么在 GPU 上并行

逐 token 执行一个小矩阵更新，会产生很长依赖链，难以使用大 GEMM。论文把长度为 C 的 chunk 内递归改写为矩阵操作，chunk 之间只传递 state。[S1，§3.1]

下面用更直接的残差记法推导，帮助对应公式，而不是建议实际 kernel 显式求逆。

## 5.1 第一步：累计衰减

在一个 chunk 内，定义向量 $\gamma_i = \prod_{r=1}^{i} \alpha_r$。矩阵 $\Gamma$ 按行堆叠这些向量，$B = \mathrm{Diag}(\beta_1, \ldots, \beta_C)$，Q/K/V 按行堆叠 token。

定义加权 keys：

$$K_a = \Gamma \odot K, \qquad K_b = K \oslash \Gamma,$$

其中 $\odot$ 和 $\oslash$ 是逐元素乘除。设 $R \in \mathbb{R}^{C \times d_v}$ 的每一行是含 $\beta$ 的纠错写入残差。

## 5.2 第二步：将内部递归变成下三角系统

在时刻 i，旧 state 对当前 key 的预测，由进入 chunk 的 $S_0$ 与前面各步写入共同组成。因此

$$R = BV - BK_a S_0 - BLR, \qquad L = \mathrm{StrictTril}(K_a K_b^T).$$

整理为

$$(I + BL)R = BV - BK_a S_0.$$

令

$$U = (I + BL)^{-1}BV, \qquad W = (I + BL)^{-1}BK_a,$$

则

$$R = U - WS_0.$$

这与论文 UT transform 中 U/W 和 pseudo-value 的作用一致。[S1，式 6–9] 实现利用三角结构的专门算法和融合，不需要调用通用 dense matrix inverse。

### 5.3 第三步：一次得到 chunk 内输出

$$A = \mathrm{Tril}\left((\Gamma \odot Q)K_b^T\right),$$

$$O = (\Gamma \odot Q)S_0 + AR.$$

第一项读取前面 chunks 留下的状态；第二项处理当前 chunk 内的历史贡献。下三角包括对角线，因为输出使用的是当前 token 写入之后的 state。

chunk 末尾状态为

$$S_C = \mathrm{Diag}(\gamma_C)S_0 + \left(K \odot \frac{\gamma_C}{\Gamma}\right)^T R.$$

分式按行广播。这样将大量逐 token 小操作转为可使用 Tensor Core 的矩阵运算，代价是 chunk 内的三角处理与额外中间量。C 过小会浪费吞吐，C 过大又提高中间存储和数值压力。

**chunk 内并行不意味着所有 chunk 没有依赖**。跨 chunk 仍需正确传递 state；context parallel 切段也必须解决这一依赖。

## 6 数值难点与 K3 的后续修改

累计衰减 $\gamma_i$ 可能很小，直接求 $K/\Gamma$ 会溢出。原始 Kimi Linear 用 log-space 处理和二级 tile 等方式控制数值范围，对角块仍可能需要更细的位置对计算。[S1，§3.2、§6.2]

K3 将 log-decay 参数化从无下界的负 softplus 改成

$$g = -5\,\mathrm{sigmoid}(e^A z), \qquad \alpha = e^g.$$

16-token tile 的倒数范围因而可控，使更多对角计算也能落到 Tensor Core GEMM。K3 还将输

出 gate 改为全秩。这些是后续架构改动，不能把它们与 2025 年原始 Kimi Linear 的 kernel 或评测混用。[S2，§2.1.1]

对于复现，至少应有三个参照：逐 token FP32、原始 chunkwise、目标低精度 kernel。分别检查 forward、state、dQ/dK/dV/dgate 和跨 chunk 边界。只检查最终分类分数，无法定位门函数顺序、下三角对角线或数值缩放的实现错误。

## 7 混合架构为何仍保留 MLA

有限 state 可以高效压缩历史，却不保证无损保存任意多的独立事实。对很多相似 key、精确数字、稀有字符串和很远的多跳检索，压缩可能丢失信息。少量 MLA 层继续保留全局内容访问，因此 3:1 混合是表达与成本的折中，而不是承认线性注意力 “失败”。[S1，§1、§4]

NoPE 的 MLA 与递归的顺序敏感性配合。没有显式 RoPE，不等于模型完全不含位置信息；递归顺序、遗忘门与短卷积已经引入时间方向。

此外，改变模型架构需要重新训练或适当的转换训练。论文摘要中的 drop-in replacement 应理解为架构替代方案，而不是在任意已训练 softmax checkpoint 上直接替换算子就保持输出。

## 8 显存与速度：75% 和 6× 该怎么理解

### 8.1 75% KV 降低不等于总显存降低 75%

若四分之三层由 KDA 替代 MLA，则长上下文下主要增长的 KV 可以减少到约四分之一。但还存在 KDA state、卷积历史、模型权重、临时 activation 和剩余 MLA KV。

$$M_{\text{hybrid}} \approx n_{\text{MLA}} Lcs + n_{\text{KDA}} h d_k d_v s_S + M_{\text{conv}} + M_{\text{other}}.$$

这是容量估算。教学例子中，一个 head 的 128×128 FP32 state 为 64 KiB；16 个 head 就是每层每请求 1 MiB，还要乘 batch 与 KDA 层数。状态与 L 无关，不代表在高并发下可以忽略。

### 8.2 6× 与 batch=1 的图不是同一个口径

论文首页 Fig.1 展示 1M 上下文下 1.84ms 与 11.48ms、约 6.3×，并说明更小的缓存允许更大的 batch。正文 Fig.7 则明确使用 **batch size=1**；其 1M decode TPOT 图示约 2.2×，prefill 图示约 2.9×。[S1，Fig.1、Fig.7、§5.6]

因此，不能把摘要的 “up to6×” 改写成任意单请求都快 6 倍。混合架构的收益既来自每请求计算变化，也来自节省缓存后能够选择更大的吞吐配置。两种收益都真实有意义，但比较条件不同。

固定 batch 对比回答的是 “同样并发下有多快”；等内存／等 SLA 的最优 batch 对比回答的是 “整套服务最多能处理多少工作”。报告时应分别列出。

## 9 质量实验：并非每一列都领先

下表摘自相同 1.4T 训练预算的长上下文比较，评测长度 128K。[S1，Table5]

| 模型 | RULER | LongBench V2 | Frames | RepoQA | 总体均分 |
|---|---|---|---|---|---|
| MLA | 81.3 | 36.1 | 60.5 | 63.0 | 52.2 |
| GDN-H | 80.5 | 32.6 | 58.7 | 63.0 | 51.2 |
| Kimi Linear + RoPE | 78.8 | 35.4 | 59.9 | 66.5 | 51.8 |
| Kimi Linear | 84.3 | 35.0 | 58.8 | 68.5 | 54.5 |

“总体均分” 沿用原论文完整任务集合的 Avg，本表只摘录其中四个任务，不是这四列的算术平均。Kimi Linear 的总体均分更高，但 MLA 在 LongBench V2 和 Frames 更好。这说明混合线性注意力是一种有竞争力的系统与质量折中，不是所有能力维度的严格支配关系。

RoPE 变体的表现也说明，位置设计与混合结构必须配套；不能认为把在普通 Transformer 中有效的部件直接加回来，总能更好。[S1]

论文还给出数学 RL 的同算法、同超参数对比，观察到 Kimi Linear 更好的学习曲线。但这是特定数学训练任务和实验设置，不自动证明所有 agentic RL 或所有 rollout 长度下都具有同样优势。

## 10 从状态缓存到推测解码：新的运行时约束

full attention 可以追加 KV 页；KDA 则更新一个 state。若推测解码验证了多个 draft、只接受前几个，系统必须恢复接受前缀对应的 state，而不是保留包含已拒绝 token 的最新状态。

可选择在候选边界保存 state、记录足够的更新中间量，或从较早 checkpoint 重放。每种方法都消耗时间或内存。由于衰减和纠错可能不可稳定逆转，不能默认从最终 state 直接 “减掉被拒绝 token” 就能恢复。

同样，prefix cache 命中、请求分叉、长会话暂停和迁移，需要同时管理 MLA KV、KDA state 与短卷积历史。这个复合状态的版本和边界，必须精确对应同一 token 前缀。

以上是依据模型递归推导的系统约束，不是声称原论文已经实现并验证所有部署方案。

## 11 面向算子开发的复现与研究计划

先用短序列构造 FP32 recurrent reference，测试纯衰减、零写入、重复 key、正交 key、chunk 长度不能整除、变长 batch、非零初始 state 等情形。检查每一步 state，而不只最后输出。

随后实现小 C 的三角公式，与 recurrent 对照，验证包含对角的 causal mask 和末尾状态传递。最后再引入 Tensor Core、融合、低精度、不同 tile 与门函数参数化。对每个加速开关记录误差和时间，避免一次引入太多变化无法归因。

| 测量层级 | 核心指标 |
| --- | --- |
| 单算子 | forward/backward 延迟、显存流量、state 误差 |
| 跨 chunk | 边界一致性、长序列误差积累、重算成本 |
| 混合模型 | 等 batch TPOT、prefill、峰值缓存、质量 |
| 服务系统 | 等 SLA 吞吐、prefix 恢复、draft 回滚成本 |

研究切口可以是门值分布感知的数值策略、确定性 chunk 调度，或 state/KV 混合缓存。真正的贡献应同时证明正确性与实际收益，而不是只给出一个比逐 token Python 循环更快的 kernel。

## 12 最后总结与原始资料

KDA 的核心路线是：**把关联记忆的追加变成纠错，把统一遗忘变成通道级遗忘，再把串行更新改写成可高效执行的块内计算**。混合 MLA 提供部分显式全局访问，避免完全依赖固定状态。

与 HiLS 的区别在于历史表示：HiLS 选择性读取原始远处块，KDA 把历史写入有限 state。与 HiSparse 的区别在于职责：HiSparse 管理 KV 在不同存储层的位置，而不替换注意力机制。这三条路线可以形成不同系统组合，但不应混称同一种 “压缩 KV”。

**[S1] Kimi Linear v2**。 arXiv:2510.26692v2。重点：§2–4、Table5、Fig.1/7、§6.2。本文 chunk 推导采用与原文等价的残差记法，便于阅读。

**[S2] Kimi K3 v2**。 arXiv:2607.24653v2，§2.1.1、§5.1。仅用于区分后续数值参数化与运行时设计。

**[S3] 模型与实现入口**。 MoonshotAI/Kimi-Linear；Flash Linear Attention 的 KDA 算子。当前源码可能晚于论文，应固定提交后再复现。

本报告没有训练模型或执行 GPU 性能测试。二维算例、状态容量计算、稳定性解释与回滚约束是分析性推导，不是论文额外实验。